FACULDADE DE TECNOLOGIA DE INDAIATUBA

CURSO SUPERIOR DE TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTHUR DAVI GOMES

GUSTAVO SILVA DE CARVALHO

MATHEUS HENRIQUE VIEIRA NEVES

RAUL TENORIO

Saúde Unificada: documentação do Projeto Integrador

Modelo final – versão 1.0.2

Indaiatuba

2026

FACULDADE DE TECNOLOGIA DE INDAIATUBA

CURSO SUPERIOR DE TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTHUR DAVI GOMES

GUSTAVO SILVA DE CARVALHO

MATHEUS HENRIQUE VIEIRA NEVES

RAUL TENORIO

Saúde Unificada: documentação do Projeto Integrador

Modelo final – versão 1.0.2

Documentação apresentada ao Projeto Integrador do 6º semestre do Curso Superior de Tecnologia em Desenvolvimento de Software Multiplataforma da Faculdade de Tecnologia de Indaiatuba, como parte das avaliações das disciplinas de Computação em Nuvem II e Qualidade e Teste de Software, elaborado sob a orientação do(a) Prof.(a) [nome do orientador].

Indaiatuba

2026

RESUMO

A plataforma Saúde Unificada centraliza, em um prontuário identificado pelo CPF, os dados de saúde de pacientes, com acesso por site, aplicativo e uma API implantada na nuvem da AWS. Esta documentação reúne o plano e os casos de teste do sistema e a análise de escalabilidade e custo da infraestrutura. Cada funcionalidade foi descrita como um processo, com entradas e saídas esperadas, e verificada por 52 casos de teste, 41 deles automatizados; todos passaram, um com ressalva. Na nuvem, testes de carga em três rodadas mostraram que a API escala de uma para três instâncias sob carga e volta a uma sem carga, com vazão proporcional ao número de instâncias, e revelaram um defeito de conexão com o balanceador, corrigido e confirmado com zero erros. A partir da capacidade medida e de preços públicos de referência, estimou-se o custo para diferentes portes, até a rede hospitalar de todo o Brasil, com 1,5 a 2,5 milhões de atendimentos por dia: de R$ 30 a 47 mil por mês em infraestrutura e de R$ 317 a 374 mil por mês na operação realista, abaixo de um centavo por atendimento. Em dez anos, arquivar no S3 Glacier os dados com mais de 2,5 anos reduz em 44% a 46% o custo de dados, sem descartar o prontuário, que a lei exige guardar por pelo menos 20 anos, e a nuvem mostrou-se mais barata que uma infraestrutura própria.

Palavras-chave: testes de software; computação em nuvem; prontuário eletrônico.

ABSTRACT

The Saúde Unificada platform centralizes patient health records identified by the Brazilian CPF, accessed through a website, a mobile app and an API deployed on AWS. This documentation brings together the system test plan and test cases and the scalability and cost analysis of the infrastructure. Each feature was described as a process, with expected inputs and outputs, and verified by 52 test cases, 41 of them automated; all passed, one with a caveat. In the cloud, load tests in three rounds showed that the API scales from one to three instances under load and back to one without load, with throughput proportional to the number of instances, and revealed a connection defect with the load balancer, fixed and confirmed with zero errors. From the measured capacity and public reference prices, the cost was estimated for different scales, up to the entire Brazilian hospital network, with 1.5 to 2.5 million visits per day: BRL 30 to 47 thousand per month in infrastructure and BRL 317 to 374 thousand per month in a realistic operation, below one cent per visit. Over ten years, archiving data older than 2.5 years in S3 Glacier reduces data costs by 44% to 46% without deleting medical records, which Brazilian law requires to be kept for at least 20 years, and the cloud proved cheaper than an on-premises infrastructure.

Keywords: software testing; cloud computing; electronic health record.

LISTA DE GRÁFICOS

Gráfico 1 – CPU média e tarefas saudáveis durante a rodada 2.....40
Gráfico 2 – Taxa de erro por rodada e cenário.....42
Gráfico 3 – Capacidade de um processo da API por operação.....43
Gráfico 4 – Custo mensal de infraestrutura por porte.....44
Gráfico 5 – Custo mensal de banco e arquivos ao longo de dez anos.....48
Gráfico 6 – Composição do custo mensal no décimo ano, com arquivamento.....49
Gráfico 7 – Custo total em dez anos por tipo de infraestrutura.....50

LISTA DE QUADROS

Quadro 1 – Tipos de teste e ferramentas utilizadas.....12
Quadro 2 – Caso de teste CT01 – Autenticar profissional, teste 1.....13
Quadro 3 – Caso de teste CT02 – Autenticar profissional, teste 2.....14
Quadro 4 – Caso de teste CT03 – Autenticar profissional, teste 3.....14
Quadro 5 – Caso de teste CT04 – Autenticar profissional, teste 4.....14
Quadro 6 – Caso de teste CT05 – Autenticar profissional, teste 5.....14
Quadro 7 – Caso de teste CT06 – Autenticar paciente, teste 1.....16
Quadro 8 – Caso de teste CT07 – Autenticar paciente, teste 2.....16
Quadro 9 – Caso de teste CT08 – Autenticar paciente, teste 3.....16
Quadro 10 – Caso de teste CT09 – Autenticar paciente, teste 4.....16
Quadro 11 – Caso de teste CT10 – Consultar prontuário unificado, teste 1.....18
Quadro 12 – Caso de teste CT11 – Consultar prontuário unificado, teste 2.....18
Quadro 13 – Caso de teste CT12 – Consultar prontuário unificado, teste 3.....18
Quadro 14 – Caso de teste CT13 – Consultar prontuário unificado, teste 4.....18
Quadro 15 – Caso de teste CT14 – Consultar prontuário unificado, teste 5.....19
Quadro 16 – Caso de teste CT15 – Cadastrar paciente, teste 1.....19
Quadro 17 – Caso de teste CT16 – Cadastrar paciente, teste 2.....20
Quadro 18 – Caso de teste CT17 – Registrar atendimento, teste 1.....21
Quadro 19 – Caso de teste CT18 – Registrar atendimento, teste 2.....21
Quadro 20 – Caso de teste CT19 – Registrar atendimento, teste 3.....21
Quadro 21 – Caso de teste CT20 – Registrar atendimento, teste 4.....22
Quadro 22 – Caso de teste CT21 – Registrar atendimento, teste 5.....22
Quadro 23 – Caso de teste CT22 – Registrar atendimento, teste 6.....22
Quadro 24 – Caso de teste CT23 – Registrar atendimento, teste 7.....22
Quadro 25 – Caso de teste CT24 – Anexar e baixar documento, teste 1.....24
Quadro 26 – Caso de teste CT25 – Anexar e baixar documento, teste 2.....24
Quadro 27 – Caso de teste CT26 – Anexar e baixar documento, teste 3.....24
Quadro 28 – Caso de teste CT27 – Anexar e baixar documento, teste 4.....24
Quadro 29 – Caso de teste CT28 – Anexar e baixar documento, teste 5.....25
Quadro 30 – Caso de teste CT29 – Verificar autenticidade de documento, teste 1. .26
Quadro 31 – Caso de teste CT30 – Verificar autenticidade de documento, teste 2. .26
Quadro 32 – Caso de teste CT31 – Controlar privacidade, teste 1.....27

Quadro 33 – Caso de teste CT32 – Controlar privacidade, teste 2.....27
Quadro 34 – Caso de teste CT33 – Controlar privacidade, teste 3.....28
Quadro 35 – Caso de teste CT34 – Controlar privacidade, teste 4.....28
Quadro 36 – Caso de teste CT35 – Controlar privacidade, teste 5.....28
Quadro 37 – Caso de teste CT36 – Consultar dados no aplicativo, teste 1.....29
Quadro 38 – Caso de teste CT37 – Consultar dados no aplicativo, teste 2.....29
Quadro 39 – Caso de teste CT38 – Consultar dados no aplicativo, teste 3.....30
Quadro 40 – Caso de teste CT39 – Consultar dados no aplicativo, teste 4.....30
Quadro 41 – Caso de teste CT40 – Consultar dados no aplicativo, teste 5.....30
Quadro 42 – Caso de teste CT41 – Gerar painel regional, teste 1.....31
Quadro 43 – Caso de teste CT42 – Gerar painel regional, teste 2.....31
Quadro 44 – Caso de teste CT43 – Gerar painel regional, teste 3.....31
Quadro 45 – Caso de teste CT44 – Gerar painel regional, teste 4.....32
Quadro 46 – Caso de teste CT45 – Proteger e publicar a API na nuvem, teste 1.....33
Quadro 47 – Caso de teste CT46 – Proteger e publicar a API na nuvem, teste 2.....33
Quadro 48 – Caso de teste CT47 – Proteger e publicar a API na nuvem, teste 3.....33
Quadro 49 – Caso de teste CT48 – Proteger e publicar a API na nuvem, teste 4.....33
Quadro 50 – Caso de teste CT49 – Proteger e publicar a API na nuvem, teste 5.....34
Quadro 51 – Caso de teste CT50 – Escalar a API sob carga, teste 1.....35
Quadro 52 – Caso de teste CT51 – Escalar a API sob carga, teste 2.....35
Quadro 53 – Caso de teste CT52 – Escalar a API sob carga, teste 3.....35
Quadro 54 – Defeitos encontrados nos testes.....36
Quadro 55 – Componentes da infraestrutura na nuvem.....37
Quadro 56 – Premissas de uso para a estimativa por porte.....43
Quadro 57 – Possibilidade de descarte por tipo de dado.....46
Quadro 58 – Local de armazenamento conforme a idade do dado.....47
Quadro 59 – Premissas da simulação de dez anos.....47
Quadro 60 – Componentes de uma infraestrutura própria.....49

LISTA DE TABELAS

Tabela 1 – Resultado da execução por processo.....36
Tabela 2 – Cenários dos testes de carga na AWS.....38
Tabela 3 – Resultados dos testes de carga por rodada.....40
Tabela 4 – Linha do tempo do escalonamento automático (horário de Brasília).....41
Tabela 5 – Capacidade medida em computador local e estimada para uma tarefa na AWS.....43
Tabela 6 – Custo mensal estimado da infraestrutura por porte (US$).....44
Tabela 7 – Requisições geradas por atendimento.....45
Tabela 8 – Volume e capacidade necessários no cenário nacional.....45
Tabela 9 – Custo mensal estimado no cenário nacional (US$).....45
Tabela 10 – Evolução anual no cenário de 1,5 milhão de atendimentos por dia.....47
Tabela 11 – Evolução anual no cenário de 2,5 milhões de atendimentos por dia.....48
Tabela 12 – Custo total de banco e arquivos em dez anos.....49
Tabela 13 – Custo total em dez anos: nuvem e infraestrutura própria (R$ milhões). 50
Tabela 14 – Arquivos em infraestrutura própria e na nuvem em dez anos (modelo híbrido).....51
Tabela 15 – Infraestrutura isolada e operação realista na nuvem.....51

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ALB	Application Load Balancer
API	Application Programming Interface (interface de programação de aplicações)
AWS	Amazon Web Services
CDK	Cloud Development Kit
CDN	Content Delivery Network (rede de distribuição de conteúdo)
CFM	Conselho Federal de Medicina
CORS	Cross-Origin Resource Sharing (compartilhamento entre origens)
CPF	Cadastro de Pessoas Físicas
CPU	Central Processing Unit (unidade central de processamento)
CRM	Conselho Regional de Medicina
DDoS	Distributed Denial of Service (ataque distribuído de negação de serviço)
ECS	Elastic Container Service
FHIR	Fast Healthcare Interoperability Resources
HL7	Health Level Seven
HTTPS	Hypertext Transfer Protocol Secure
ICP-Brasil	Infraestrutura de Chaves Públicas Brasileira
JSON	JavaScript Object Notation
JWT	JSON Web Token
LGPD	Lei Geral de Proteção de Dados Pessoais
PB	Petabyte
PDF	Portable Document Format
S3	Simple Storage Service
SHA-256	Secure Hash Algorithm de 256 bits
SSD	Solid State Drive
TB	Terabyte
vCPU	Virtual CPU (processador virtual)
WAF	Web Application Firewall

SUMÁRIO

1 INTRODUÇÃO.....11
2 PLANO E CASOS DE TESTE.....12
2.1 Estratégia de testes.....12
2.2 Processo: Autenticar profissional.....12
2.3 Processo: Autenticar paciente.....15
2.4 Processo: Consultar prontuário unificado.....17
2.5 Processo: Cadastrar paciente.....19
2.6 Processo: Registrar atendimento.....20
2.7 Processo: Anexar e baixar documento.....23
2.8 Processo: Verificar autenticidade de documento.....25
2.9 Processo: Controlar privacidade.....26
2.10 Processo: Consultar dados no aplicativo.....28
2.11 Processo: Gerar painel regional.....30
2.12 Processo: Proteger e publicar a API na nuvem.....32
2.13 Processo: Escalar a API sob carga.....34
2.14 Resultados da execução dos testes.....35
3 ARQUITETURA E FUNCIONAMENTO DO BALANCEAMENTO.....37
3.1 Arquitetura implantada na AWS.....37
3.2 Balanceamento de carga e escalonamento automático.....37
4 PROCEDIMENTOS METODOLÓGICOS.....38
4.1 Testes de carga.....38
4.2 Medição da capacidade por operação.....38
4.3 Estimativa de custos.....39
5 RESULTADOS E DISCUSSÃO.....40
5.1 Testes de carga na AWS.....40
5.2 Defeito de conexão encontrado e corrigido.....41
5.3 Distribuição desigual da carga.....42
5.4 Capacidade por operação.....42
5.5 Custo conforme o porte.....43
5.6 Cenário de abrangência nacional.....44
5.7 Banco de dados e arquivos ao longo de dez anos.....46
5.7.1 O que a legislação permite descartar.....46

5.7.2 Política de armazenamento por idade do dado.....46
5.7.3 Resultados da simulação.....47
5.8 Infraestrutura própria em comparação com a nuvem.....49
5.9 Realismo da estimativa.....51
6 CONSIDERAÇÕES FINAIS.....53
REFERÊNCIAS.....54

1 Introdução

A plataforma Saúde Unificada reúne, em um prontuário único identificado pelo CPF, as informações de saúde de cada paciente: alergias, medicamentos em uso, cirurgias, exames, condições clínicas e o histórico de atendimentos. O sistema é composto por um site para profissionais de saúde e gestores públicos, um aplicativo para pacientes e uma API compartilhada, implantada na nuvem da *Amazon Web Services* (AWS).

Para garantir que cada funcionalidade se comporta como especificado, elaborou-se um plano de testes. Seguindo o modelo de Pressman e Maxim (2016), cada funcionalidade é descrita como um processo, com as entradas que recebe e as saídas que produz em cada situação. A partir dessa descrição, definem-se os testes, com a entrada e a saída esperada, e cada teste é detalhado em um caso de teste, com nome, pré-condição, descrição, dados de entrada e resultado.

Além do comportamento funcional, um sistema desse tipo precisa continuar respondendo quando o número de usuários cresce e manter o custo sob controle. Por isso, busca-se também responder a duas perguntas: o balanceamento de carga e o escalonamento automático (*auto scaling*) implantados funcionam na prática? E quanto custaria operar a plataforma se ela fosse adotada em escala, chegando à rede hospitalar de todo o Brasil?

Os objetivos são: descrever os processos do sistema e verificar, por meio de casos de teste automatizados e manuais, se cada um produz as saídas esperadas; verificar, com testes de carga na AWS, se a API distribui as requisições entre várias instâncias e aumenta ou reduz a quantidade de instâncias conforme a demanda; medir a capacidade de cada instância e estimar o custo mensal e em dez anos da infraestrutura para diferentes portes, comparando a nuvem com uma infraestrutura própria.

Trata-se de uma pesquisa aplicada, de caráter quantitativo, baseada em testes controlados e em simulação de custos a partir de premissas declaradas. Além desta introdução, o documento apresenta o plano e os casos de teste (seção 2), a arquitetura e o funcionamento do balanceamento (seção 3), os procedimentos metodológicos (seção 4), os resultados e a discussão (seção 5) e as considerações finais (seção 6).

2 Plano e casos de teste

2.1 Estratégia de testes

Os testes cobrem 12 processos do sistema, em 52 casos de teste: 41 automatizados, executados a cada alteração do código no GitHub Actions, e 11 manuais, executados no navegador ou na AWS. O Quadro 1 resume os tipos de teste e as ferramentas.

Quadro 1 – Tipos de teste e ferramentas utilizadas

Tipo	Como é executado	Ferramenta
Funcional automatizado	A API completa é iniciada com um banco de dados em memória e recebe requisições reais	Vitest, Supertest e mongodb-memory-server
Unitário	Funções isoladas, como a validação de CPF e de senha	Vitest
Manual (site)	Telas usadas no navegador, com o site ligado à API	Navegador
Segurança	Cabeçalhos, CORS, permissões por perfil e limite de tentativas	Vitest e Supertest
Nuvem e carga	API publicada na AWS e carga simulada de usuários	Script próprio e Amazon CloudWatch

Fonte: Elaborado pelos autores (2026).

Em cada processo a seguir, apresentam-se a descrição do processo, o plano de testes, com a entrada e a saída esperada de cada teste, e as fichas dos casos de teste. Na descrição, os termos entre os sinais < e > indicam a informação que o processo devolve em cada situação.

2.2 Processo: Autenticar profissional

Autenticar Profissional realiza o login de médicos e gestores no site da Saúde Unificada. O processo recebe o e-mail e a senha digitados no formulário de login. O e-mail é buscado no cadastro de usuários e a senha é comparada com o resumo criptográfico (*hash* bcrypt) armazenado. Se a senha coincide, <token de acesso e dados do usuário> são passados para o site, que redireciona o usuário para a área do seu perfil: médico ou gestor. Se o e-mail não existe ou a senha não coincide, <mensagem = "E-mail ou senha inválidos."> é devolvida, a mesma nos dois casos, para não revelar quais contas existem. Se houver mais de cinco tentativas em um

minuto para a mesma conta, <mensagem = "Muitas tentativas de entrada."> é devolvida até o fim do bloqueio, mesmo com a senha correta.

Plano de testes

Teste1: entrada de e-mail e senha válidos de um médico

Entrada1: e-mail = medico@teste.com; senha = 123456

Saída1: token de acesso e perfil = médico

Teste2: entrada de senha incorreta

Entrada2: e-mail = medico@teste.com; senha = errada

Saída2: mensagem = "E-mail ou senha inválidos."

Teste3: entrada de e-mail não cadastrado

Entrada3: e-mail = ninguem@teste.com; senha = 123456

Saída3: mensagem = "E-mail ou senha inválidos."

Teste4: seis tentativas seguidas em um minuto

Entrada4: 6 logins com senha errada para a mesma conta

Saída4: mensagem = "Muitas tentativas de entrada."

Teste5: login pelo site ligado à API

Entrada5: e-mail = medico@teste.com; senha = 123456 (no navegador)

Saída5: tela "Meu registro" com dados da API

Casos de teste

Quadro 2 – Caso de teste CT01 – Autenticar profissional, teste 1

Nome:	Entrada de e-mail e senha válidos de um médico
Pré-condição:	Usuário médico cadastrado; formulário de login do site aberto.
Descrição do teste:	O usuário digita o e-mail e a senha e clica em Entrar. A senha é comparada com o hash armazenado.
Dados de entrada:	E-mail e senha corretos de um médico cadastrado.
Resultado:	Sistema valida as credenciais, devolve o token (código 200) e redireciona para a área do médico.
Resultado obtido:	Código 200, token JWT e perfil médico; redirecionado para "Meu registro". (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 3 – Caso de teste CT02 – Autenticar profissional, teste 2

Nome:	Entrada de senha incorreta
Pré-condição:	Usuário médico cadastrado; formulário de login aberto.
Descrição do teste:	O usuário digita o e-mail correto e uma senha diferente da cadastrada.
Dados de entrada:	E-mail cadastrado e senha incorreta.
Resultado:	Sistema recusa o acesso (código 401) com a mensagem genérica.
Resultado obtido:	Código 401 com a mensagem esperada. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 4 – Caso de teste CT03 – Autenticar profissional, teste 3

Nome:	Entrada de e-mail não cadastrado
Pré-condição:	E-mail inexistente no cadastro.
Descrição do teste:	O usuário digita um e-mail que não existe no sistema.
Dados de entrada:	E-mail não cadastrado.
Resultado:	Mesma resposta do teste com senha incorreta, sem revelar que a conta não existe.
Resultado obtido:	Mensagem idêntica à da senha incorreta. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 5 – Caso de teste CT04 – Autenticar profissional, teste 4

Nome:	Seis tentativas seguidas em um minuto
Pré-condição:	Limite de tentativas ligado (5 por minuto por conta).
Descrição do teste:	O usuário erra a senha várias vezes seguidas e depois tenta a senha correta.
Dados de entrada:	Seis tentativas em menos de um minuto.
Resultado:	A partir da sexta tentativa, o sistema bloqueia (código 429), inclusive para a senha correta.
Resultado obtido:	Código 429 com a mensagem de bloqueio. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 6 – Caso de teste CT05 – Autenticar profissional, teste 5

Nome:	Login pelo site ligado à API
Pré-condição:	API em execução e site configurado para usá-la (VITE_USE_MOCK=false).
Descrição do teste:	O usuário abre a página de login no navegador, preenche o formulário e entra.
Dados de entrada:	Credenciais de um médico do seed de demonstração.
Resultado:	Sistema valida a senha e mostra a área do médico com os dados reais da API.
Resultado obtido:	Tela exibida com 4 próximos atendimentos, 4 realizados e 5 pacientes. (manual; passou)

Fonte: Elaborado pelos autores (2026).

2.3 Processo: Autenticar paciente

Autenticar Paciente realiza o acesso do paciente ao aplicativo, no estilo do gov.br. No login, o processo recebe o CPF, com ou sem máscara, e a senha; o CPF é normalizado e a senha é comparada com a armazenada. Se coincide, <token de acesso e perfil = paciente> é devolvido ao aplicativo. No primeiro acesso, o processo recebe o CPF, a data de nascimento e a nova senha. Se o CPF e a data coincidem com um prontuário existente e a senha é forte, a conta é criada e <token de acesso> é devolvido. Se não coincidem, <mensagem de cadastro não encontrado> é devolvida, sem informar qual dado está errado.

Plano de testes

Teste1: login do paciente com CPF e senha válidos
Entrada1: CPF = 229.169.264-02; senha = paciente123
Saída1: token de acesso e perfil = paciente
Teste2: primeiro acesso com dados corretos e senha forte
Entrada2: CPF da Ana; data de nascimento = 1996-11-05; senha = meuApp2026
Saída2: conta criada e token de acesso
Teste3: primeiro acesso com senha fraca
Entrada3: senha = 12345678 ou senha contendo o CPF
Saída3: mensagem com o motivo da recusa
Teste4: paciente tenta entrar pelo login do site
Entrada4: login por e-mail com a conta de paciente
Saída4: acesso negado

Casos de teste

Quadro 7 – Caso de teste CT06 – Autenticar paciente, teste 1

Nome:	Login do paciente com CPF e senha válidos
Pré-condição:	Paciente com conta no aplicativo.
Descrição do teste:	O paciente digita o CPF com máscara e a senha no aplicativo.
Dados de entrada:	CPF com máscara e senha corretos.
Resultado:	Sistema aceita o CPF com ou sem máscara e devolve o token do paciente (código 200).
Resultado obtido:	Código 200 com perfil paciente e o CPF. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 8 – Caso de teste CT07 – Autenticar paciente, teste 2

Nome:	Primeiro acesso com dados corretos e senha forte
Pré-condição:	Paciente com prontuário, mas sem conta no aplicativo.
Descrição do teste:	O paciente informa CPF, data de nascimento e uma senha nova.
Dados de entrada:	CPF e data que coincidem com o prontuário; senha com letras e números.
Resultado:	Sistema cria a conta (código 201) e o paciente já entra.
Resultado obtido:	Código 201 e acesso liberado. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 9 – Caso de teste CT08 – Autenticar paciente, teste 3

Nome:	Primeiro acesso com senha fraca
Pré-condição:	Paciente sem conta no aplicativo.
Descrição do teste:	O paciente tenta criar a conta com uma senha fraca.
Dados de entrada:	Senha só com números, comum ou contendo o CPF ou a data de nascimento.
Resultado:	Sistema recusa (código 400) e explica o motivo.
Resultado obtido:	Código 400 com o motivo para cada tipo de senha fraca. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 10 – Caso de teste CT09 – Autenticar paciente, teste 4

Nome:	Paciente tenta entrar pelo login do site
Pré-condição:	Paciente com conta no aplicativo.
Descrição do teste:	O paciente tenta usar o formulário de login dos profissionais.
Dados de entrada:	Credenciais de paciente no login por e-mail.
Resultado:	Sistema recusa (código 401): o paciente só entra pelo aplicativo, com o CPF.
Resultado obtido:	Código 401. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

2.4 Processo: Consultar prontuário unificado

Consultar Prontuário recebe o CPF do paciente digitado pelo médico na busca do site. O CPF é validado pelos dígitos verificadores. Se o CPF é inválido, <mensagem = "CPF inválido."> é devolvida. Se o médico foi bloqueado pelo paciente, <mensagem de acesso bloqueado> é devolvida. Caso contrário, o prontuário é buscado e <prontuário completo> é passado ao site: tipo sanguíneo, alergias, medicamentos, cirurgias, exames, condições clínicas e o resumo dos atendimentos. Cada consulta é registrada na trilha de auditoria (quem, o quê e quando), conforme a LGPD. O gestor não tem acesso a prontuários individuais.

Plano de testes

Teste1: médico abre o prontuário de um paciente
Entrada1: CPF = 22916926402
Saída1: prontuário completo
Teste2: registro do acesso na auditoria
Entrada2: abertura de um prontuário
Saída2: registro de acesso na auditoria
Teste3: busca com CPF inválido
Entrada3: CPF com 11 dígitos iguais ou dígito verificador errado
Saída3: mensagem de CPF inválido
Teste4: gestor tenta abrir um prontuário
Entrada4: token de gestor; CPF válido
Saída4: acesso negado
Teste5: prontuário exibido no site
Entrada5: abrir a paciente Maria no site
Saída5: seções, orientações e códigos de verificação na tela

Casos de teste

Quadro 11 – Caso de teste CT10 – Consultar prontuário unificado, teste 1

Nome:	Médico abre o prontuário de um paciente
Pré-condição:	Médico autenticado; paciente cadastrado.
Descrição do teste:	O médico busca o paciente pelo CPF e abre o prontuário.
Dados de entrada:	CPF válido de um paciente com histórico.
Resultado:	Sistema mostra todas as seções do prontuário (código 200).
Resultado obtido:	Código 200 com alergias, remédios, cirurgias, exames, condições e resumo. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 12 – Caso de teste CT11 – Consultar prontuário unificado, teste 2

Nome:	Registro do acesso na auditoria
Pré-condição:	Médico autenticado.
Descrição do teste:	O médico abre o prontuário; em seguida, consulta-se a trilha de auditoria.
Dados de entrada:	CPF de um paciente.
Resultado:	Sistema grava um registro com o médico, a ação e a data.
Resultado obtido:	Um registro "patient.view" gravado. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 13 – Caso de teste CT12 – Consultar prontuário unificado, teste 3

Nome:	Busca com CPF inválido
Pré-condição:	Médico autenticado.
Descrição do teste:	O médico digita um CPF que não passa na validação.
Dados de entrada:	CPF inválido.
Resultado:	Sistema recusa (código 400) informando o motivo.
Resultado obtido:	Código 400. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 14 – Caso de teste CT13 – Consultar prontuário unificado, teste 4

Nome:	Gestor tenta abrir um prontuário
Pré-condição:	Gestor autenticado.
Descrição do teste:	O gestor tenta acessar o prontuário individual de um paciente.
Dados de entrada:	CPF de um paciente cadastrado.
Resultado:	Sistema nega o acesso (código 403): o gestor só vê dados agregados.
Resultado obtido:	Código 403. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 15 – Caso de teste CT14 – Consultar prontuário unificado, teste 5

Nome:	Prontuário exibido no site
Pré-condição:	Site ligado à API; médico autenticado.
Descrição do teste:	O médico abre o prontuário pelo navegador.
Dados de entrada:	Paciente do seed de demonstração.
Resultado:	Site exibe o prontuário, as orientações do próximo atendimento e o código de cada documento.
Resultado obtido:	Tudo exibido corretamente. (manual; passou)

Fonte: Elaborado pelos autores (2026).

2.5 Processo: Cadastrar paciente

Cadastrar Paciente recebe do médico os dados básicos de um paciente novo: CPF, nome completo, data de nascimento, sexo, tipo sanguíneo e região. O CPF é validado e procurado no cadastro. Se já existe, <mensagem = "Já existe um paciente cadastrado com este CPF."> é devolvida. Se os dados são válidos, o prontuário é criado vazio e <prontuário criado> é devolvido. A partir daí, o prontuário é atualizado a cada atendimento pelo profissional que atendeu.

Plano de testes

Teste1: cadastro de paciente novo

Entrada1: CPF válido inédito; nome; nascimento; tipo sanguíneo; região

Saída1: prontuário criado

Teste2: cadastro com CPF repetido

Entrada2: CPF já cadastrado

Saída2: mensagem = "Já existe um paciente cadastrado com este CPF."

Casos de teste

Quadro 16 – Caso de teste CT15 – Cadastrar paciente, teste 1

Nome:	Cadastro de paciente novo
Pré-condição:	Médico autenticado; CPF ainda não cadastrado.
Descrição do teste:	O médico preenche o formulário de cadastro e salva.
Dados de entrada:	Dados completos e válidos.
Resultado:	Sistema cria o prontuário vazio (código 201).
Resultado obtido:	Código 201. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 17 – Caso de teste CT16 – Cadastrar paciente, teste 2

Nome:	Cadastro com CPF repetido
Pré-condição:	Paciente já cadastrado.
Descrição do teste:	O médico tenta cadastrar de novo o mesmo CPF.
Dados de entrada:	CPF existente.
Resultado:	Sistema recusa (código 409) com a mensagem.
Resultado obtido:	Código 409 com a mensagem esperada. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

2.6 Processo: Registrar atendimento

Registrar Atendimento recebe do médico os dados do atendimento: paciente, data, situação (realizado ou agendado), motivo, resumo e, opcionalmente, novas alergias, medicamentos, condições clínicas e orientações para o paciente. Se o médico informado não é o médico autenticado, <mensagem de permissão negada> é devolvida. Se é um atendimento realizado com data futura, ou um agendamento com data passada, <mensagem de data inválida> é devolvida. Caso contrário, o atendimento e as atualizações do prontuário são gravados numa única transação: se qualquer parte falhar, nada é gravado. Então <atendimento registrado> é passado ao site.

Plano de testes

Teste1: consulta da agenda do médico

Entrada1: médico autenticado

Saída1: lista de atendimentos passados e próximos

Teste2: registro de atendimento com atualização do prontuário

Entrada2: resumo; nova alergia; novo medicamento; nova condição

Saída2: atendimento registrado e prontuário atualizado

Teste3: falha no meio do registro

Entrada3: agendamento de outro paciente

Saída3: erro e nada gravado

Teste4: agendamento com orientações ao paciente

Entrada4: situação = agendado; orientações = "Trazer a adrenalina..."

Saída4: agendamento com as orientações

Teste5: atendimento realizado com data futura

Entrada5: situação = realizado; data = amanhã

Saída5: mensagem de data inválida

Teste6: registro em nome de outro médico

Entrada6: médico informado = Dra. Camila (logado: Dr. Rafael)

Saída6: permissão negada

Teste7: registro pelo formulário do site

Entrada7: motivo e resumo preenchidos no navegador

Saída7: mensagem "Atendimento registrado e prontuário atualizado."

Casos de teste

Quadro 18 – Caso de teste CT17 – Registrar atendimento, teste 1

Nome:	Consulta da agenda do médico
Pré-condição:	Médico com atendimentos no seed.
Descrição do teste:	O médico abre a sua agenda.
Dados de entrada:	Identificador do próprio médico.
Resultado:	Sistema lista os atendimentos em ordem de data.
Resultado obtido:	8 atendimentos, sendo 4 agendados. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 19 – Caso de teste CT18 – Registrar atendimento, teste 2

Nome:	Registro de atendimento com atualização do prontuário
Pré-condição:	Médico autenticado; paciente cadastrado.
Descrição do teste:	O médico registra um atendimento realizado com itens novos para o prontuário.
Dados de entrada:	Atendimento completo e válido.
Resultado:	Sistema grava o atendimento (código 201) e os itens aparecem no prontuário.
Resultado obtido:	Código 201 e prontuário atualizado. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 20 – Caso de teste CT19 – Registrar atendimento, teste 3

Nome:	Falha no meio do registro
Pré-condição:	Agendamento existente de outro paciente.
Descrição do teste:	O médico tenta concluir um agendamento que pertence a outro paciente.
Dados de entrada:	Identificador de agendamento incompatível.
Resultado:	Sistema recusa (código 400) e o prontuário permanece intacto.
Resultado obtido:	Código 400; nada foi gravado. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 21 – Caso de teste CT20 – Registrar atendimento, teste 4

Nome:	Agendamento com orientações ao paciente
Pré-condição:	Médico autenticado.
Descrição do teste:	O médico agenda um retorno com orientações de preparo.
Dados de entrada:	Data futura e texto de orientações.
Resultado:	Sistema grava o agendamento com as orientações, que aparecem no aplicativo.
Resultado obtido:	Código 201 com as orientações. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 22 – Caso de teste CT21 – Registrar atendimento, teste 5

Nome:	Atendimento realizado com data futura
Pré-condição:	Médico autenticado.
Descrição do teste:	O médico tenta registrar como realizado um atendimento de amanhã.
Dados de entrada:	Data futura.
Resultado:	Sistema recusa (código 400).
Resultado obtido:	Código 400. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 23 – Caso de teste CT22 – Registrar atendimento, teste 6

Nome:	Registro em nome de outro médico
Pré-condição:	Médico autenticado.
Descrição do teste:	O médico tenta registrar um atendimento como se fosse outro profissional.
Dados de entrada:	Identificador de outro médico.
Resultado:	Sistema nega (código 403).
Resultado obtido:	Código 403. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 24 – Caso de teste CT23 – Registrar atendimento, teste 7

Nome:	Registro pelo formulário do site
Pré-condição:	Site ligado à API; ortopedista autenticado.
Descrição do teste:	O médico abre o paciente Roberto, preenche o formulário e salva.
Dados de entrada:	Motivo e resumo do atendimento.
Resultado:	Site mostra a mensagem de sucesso e o prontuário já inclui o atendimento.
Resultado obtido:	Mensagem exibida e atendimento no resumo do prontuário. (manual; passou)

Fonte: Elaborado pelos autores (2026).

2.7 Processo: Anexar e baixar documento

Anexar Documento recebe do médico um arquivo (receita, laudo, exame ou outro) ligado a um atendimento. O tipo do arquivo é conferido pelo conteúdo, e não pelo nome. Se não é PDF, JPG ou PNG, <mensagem de formato não aceito> é devolvida; se passa de 10 MB, <mensagem de arquivo muito grande> é devolvida. Caso contrário, o arquivo é gravado no armazenamento (Amazon S3) e recebe um código de verificação e o seu SHA-256; então <documento anexado> é devolvido. Baixar Documento recebe o pedido de um médico ou do próprio paciente e devolve <link temporário> válido por cinco minutos.

Plano de testes

Teste1: anexar receita em PDF
Entrada1: arquivo PDF de 1 KB; tipo = receita
Saída1: documento com código de verificação e SHA-256
Teste2: arquivo disfarçado
Entrada2: executável renomeado para .pdf
Saída2: mensagem de formato não aceito
Teste3: arquivo acima do limite
Entrada3: arquivo de 11 MB
Saída3: mensagem de arquivo muito grande
Teste4: download pelo link temporário
Entrada4: pedido de download de um documento anexado
Saída4: arquivo idêntico ao enviado
Teste5: abrir documento pelo site
Entrada5: clique em Abrir no prontuário
Saída5: PDF aberto

Casos de teste

Quadro 25 – Caso de teste CT24 – Anexar e baixar documento, teste 1

Nome:	Anexar receita em PDF
Pré-condição:	Atendimento do próprio médico.
Descrição do teste:	O médico anexa uma receita ao atendimento.
Dados de entrada:	PDF válido.
Resultado:	Sistema grava o arquivo (código 201) com código único no formato XXXX-XXXX-XXXX.
Resultado obtido:	Código 201; códigos únicos; SHA-256 confere com o arquivo. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 26 – Caso de teste CT25 – Anexar e baixar documento, teste 2

Nome:	Arquivo disfarçado
Pré-condição:	Atendimento do próprio médico.
Descrição do teste:	O médico envia um arquivo cujo conteúdo não é PDF, JPG ou PNG.
Dados de entrada:	Arquivo com extensão falsa.
Resultado:	Sistema recusa (código 400) após conferir o conteúdo.
Resultado obtido:	Código 400. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 27 – Caso de teste CT26 – Anexar e baixar documento, teste 3

Nome:	Arquivo acima do limite
Pré-condição:	Atendimento do próprio médico.
Descrição do teste:	O médico envia um arquivo maior que 10 MB.
Dados de entrada:	Arquivo de 11 MB.
Resultado:	Sistema recusa (código 413).
Resultado obtido:	Código 413. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 28 – Caso de teste CT27 – Anexar e baixar documento, teste 4

Nome:	Download pelo link temporário
Pré-condição:	Documento anexado.
Descrição do teste:	O médico pede o link e abre o arquivo por ele.
Dados de entrada:	Identificadores do atendimento e do documento.
Resultado:	O arquivo baixado é idêntico ao enviado; o link expira em 5 minutos.
Resultado obtido:	Bytes idênticos, também no S3 real da AWS. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 29 – Caso de teste CT28 – Anexar e baixar documento, teste 5

Nome:	Abrir documento pelo site
Pré-condição:	Site ligado à API; documento anexado.
Descrição do teste:	O médico clica no botão Abrir de um documento.
Dados de entrada:	Documento de um paciente do seed.
Resultado:	Site pede o link à API e abre o PDF.
Resultado obtido:	Link gerado (código 200) e PDF de 996 bytes aberto. (manual; passou)

Fonte: Elaborado pelos autores (2026).

2.8 Processo: Verificar autenticidade de documento

Verificar Documento atende quem recebe uma receita ou um laudo, como uma farmácia, sem exigir login. O processo recebe o código de verificação impresso no documento e o normaliza, aceitando letras minúsculas, ausência de traços e as letras O, I e L no lugar de 0 e 1. Se o código corresponde a um documento, <documento válido> é devolvido com o tipo, a data, o nome e o CRM do médico, as iniciais do paciente e o SHA-256 do arquivo, sem CPF nem nome completo. Se não há correspondência, <mensagem = "Nenhum documento com esse código."> é devolvida.

Plano de testes

Teste1: conferência de receita sem login

Entrada1: código de verificação de uma receita

Saída1: documento válido, só com as iniciais do paciente

Teste2: código digitado de outra forma ou inexistente

Entrada2: código em minúsculas, sem traço, com O no lugar de 0; código ZZZZ-ZZZZ-ZZZZ

Saída2: aceita a variação; inexistente não encontrado

Casos de teste

Quadro 30 – Caso de teste CT29 – Verificar autenticidade de documento, teste 1

Nome:	Conferência de receita sem login
Pré-condição:	Receita anexada com código gerado.
Descrição do teste:	A farmácia digita o código impresso na receita.
Dados de entrada:	Código existente.
Resultado:	Sistema confirma o documento com médico, CRM e data, sem expor o CPF.
Resultado obtido:	Código 200; iniciais "J. P. O. S."; nenhum CPF na resposta. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 31 – Caso de teste CT30 – Verificar autenticidade de documento, teste 2

Nome:	Código digitado de outra forma ou inexistente
Pré-condição:	Receita anexada com código gerado.
Descrição do teste:	A farmácia digita o código com variações e depois um código que não existe.
Dados de entrada:	Variações do código e código inexistente.
Resultado:	Sistema aceita a variação e devolve código 404 para o inexistente.
Resultado obtido:	Como esperado. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

2.9 Processo: Controlar privacidade

Controlar Privacidade permite ao paciente decidir quais médicos podem acessar o seu prontuário. O processo lista os médicos ligados ao paciente: quem o atendeu, quem vai atendê-lo e quem abriu o prontuário. Ao receber o pedido de bloqueio de um médico, o bloqueio é gravado; a partir daí, quando esse médico tenta abrir o prontuário, o histórico ou os documentos do paciente, ou registrar um atendimento, <mensagem = "O paciente bloqueou o seu acesso a este prontuário."> é devolvida. Os demais médicos não são afetados. O paciente pode liberar o acesso a qualquer momento.

Plano de testes

Teste1: lista de médicos ligados à paciente

Entrada1: paciente autenticada no aplicativo

Saída1: lista com o Dr. Rafael, não bloqueado

Teste2: bloqueio de um médico

Entrada2: bloquear = Dr. Rafael

Saída2: Dr. Rafael sem acesso ao prontuário, histórico, documentos e registro

Teste3: outros médicos não afetados

Entrada3: Dra. Camila abre o prontuário da paciente

Saída3: prontuário exibido

Teste4: liberação do acesso

Entrada4: liberar = Dr. Rafael

Saída4: acesso restabelecido

Teste5: bloqueio de quem não é médico

Entrada5: identificador do gestor ou inexistente

Saída5: mensagem = "Médico não encontrado."

Casos de teste

Quadro 32 – Caso de teste CT31 – Controlar privacidade, teste 1

Nome:	Lista de médicos ligados à paciente
Pré-condição:	Paciente com atendimentos registrados.
Descrição do teste:	A paciente abre a tela de privacidade.
Dados de entrada:	Token da paciente.
Resultado:	Sistema lista os médicos ligados a ela, sem o gestor.
Resultado obtido:	Dr. Rafael listado com bloqueio = falso. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 33 – Caso de teste CT32 – Controlar privacidade, teste 2

Nome:	Bloqueio de um médico
Pré-condição:	Médico ligado à paciente.
Descrição do teste:	A paciente bloqueia o médico; em seguida, ele tenta acessar os dados dela.
Dados de entrada:	Identificador do médico.
Resultado:	Todas as tentativas do médico bloqueado recebem código 403 com a mensagem.
Resultado obtido:	Código 403 em todas as rotas. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 34 – Caso de teste CT33 – Controlar privacidade, teste 3

Nome:	Outros médicos não afetados
Pré-condição:	Dr. Rafael bloqueado.
Descrição do teste:	Outro médico abre o prontuário da mesma paciente.
Dados de entrada:	Token da Dra. Camila.
Resultado:	Acesso normal (código 200).
Resultado obtido:	Código 200. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 35 – Caso de teste CT34 – Controlar privacidade, teste 4

Nome:	Liberação do acesso
Pré-condição:	Dr. Rafael bloqueado.
Descrição do teste:	A paciente libera o médico e ele tenta abrir o prontuário.
Dados de entrada:	Identificador do médico.
Resultado:	Acesso volta ao normal.
Resultado obtido:	Código 200. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 36 – Caso de teste CT35 – Controlar privacidade, teste 5

Nome:	Bloqueio de quem não é médico
Pré-condição:	Paciente autenticada.
Descrição do teste:	A paciente tenta bloquear um identificador que não é de médico.
Dados de entrada:	Identificador inválido.
Resultado:	Sistema recusa (código 404).
Resultado obtido:	Código 404. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

2.10 Processo: Consultar dados no aplicativo

Consultar Dados no Aplicativo atende o paciente autenticado, que não pode alterar as próprias informações. O CPF vem do próprio token, de modo que ninguém consegue ver os dados de outra pessoa. O processo devolve <carteira digital> com o prontuário completo, <atendimentos> separados entre o histórico e os próximos, com as orientações de preparo, e <registro de acessos>, com quem abriu o prontuário e quando. Pedidos de documentos de outro paciente recebem a mesma resposta de um documento inexistente.

Plano de testes

Teste1: carteira digital de saúde

Entrada1: paciente autenticada

Saída1: prontuário da própria paciente

Teste2: próximos atendimentos com orientações

Entrada2: filtro = agendado

Saída2: lista com as orientações de preparo

Teste3: quem acessou meus dados

Entrada3: paciente autenticada após acesso de um médico

Saída3: registro com nome do médico e data

Teste4: documento de outra pessoa

Entrada4: identificador de documento do João

Saída4: documento não encontrado

Teste5: paciente usa rota de médico

Entrada5: token de paciente em GET /patients/:cpf

Saída5: acesso negado

Casos de teste

Quadro 37 – Caso de teste CT36 – Consultar dados no aplicativo, teste 1

Nome:	Carteira digital de saúde
Pré-condição:	Paciente com conta no aplicativo.
Descrição do teste:	A paciente abre a carteira digital.
Dados de entrada:	Token da paciente.
Resultado:	Sistema devolve o próprio prontuário, somente leitura.
Resultado obtido:	Código 200. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 38 – Caso de teste CT37 – Consultar dados no aplicativo, teste 2

Nome:	Próximos atendimentos com orientações
Pré-condição:	Paciente com agendamento futuro.
Descrição do teste:	A paciente abre os próximos atendimentos.
Dados de entrada:	Token da paciente.
Resultado:	Sistema lista os agendamentos com as orientações.
Resultado obtido:	"Jejum de 8 horas..." exibido. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 39 – Caso de teste CT38 – Consultar dados no aplicativo, teste 3

Nome:	Quem acessou meus dados
Pré-condição:	Prontuário aberto por um médico.
Descrição do teste:	A paciente abre o histórico de acessos.
Dados de entrada:	Token da paciente.
Resultado:	Sistema mostra o médico e a data do acesso (LGPD).
Resultado obtido:	Registro presente. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 40 – Caso de teste CT39 – Consultar dados no aplicativo, teste 4

Nome:	Documento de outra pessoa
Pré-condição:	Paciente autenticada.
Descrição do teste:	A paciente tenta baixar um documento que não é dela.
Dados de entrada:	Identificadores de outro paciente.
Resultado:	Sistema responde como se o documento não existisse (código 404).
Resultado obtido:	Código 404. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 41 – Caso de teste CT40 – Consultar dados no aplicativo, teste 5

Nome:	Paciente usa rota de médico
Pré-condição:	Paciente autenticada.
Descrição do teste:	A paciente tenta usar uma rota exclusiva do médico.
Dados de entrada:	Token de paciente.
Resultado:	Sistema nega (código 403).
Resultado obtido:	Código 403. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

2.11 Processo: Gerar painel regional

Gerar Painel Regional atende o gestor público. O processo recebe a região e o período (mês inicial e final) escolhidos no painel. Se o mês inicial é posterior ao final ou a região não existe, <mensagem de filtro inválido> é devolvida. Caso contrário, <dados agregados> são devolvidos por região e mês: atendimentos, pacientes únicos e condições mais frequentes, sem CPF, nome ou identificador de paciente. Médicos não têm acesso ao painel.

Plano de testes

Teste1: dados por região e período

Entrada1: região = Campinas; de 2026-01 a 2026-06

Saída1: somente dados agregados

Teste2: período invertido

Entrada2: mês inicial depois do final

Saída2: mensagem de período inválido

Teste3: médico tenta abrir o painel

Entrada3: token de médico

Saída3: acesso negado

Teste4: painel no site

Entrada4: gestor@teste.com no navegador

Saída4: indicadores e gráficos com dados da API

Casos de teste

Quadro 42 – Caso de teste CT41 – Gerar painel regional, teste 1

Nome:	Dados por região e período
Pré-condição:	Gestor autenticado.
Descrição do teste:	O gestor filtra o painel por região e período.
Dados de entrada:	Filtros válidos.
Resultado:	Sistema devolve os agregados, sem dados pessoais (código 200).
Resultado obtido:	Código 200, somente agregados. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 43 – Caso de teste CT42 – Gerar painel regional, teste 2

Nome:	Período invertido
Pré-condição:	Gestor autenticado.
Descrição do teste:	O gestor escolhe o mês inicial depois do final.
Dados de entrada:	Período invertido.
Resultado:	Sistema recusa (código 400) com a mensagem.
Resultado obtido:	Código 400. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 44 – Caso de teste CT43 – Gerar painel regional, teste 3

Nome:	Médico tenta abrir o painel
Pré-condição:	Médico autenticado.
Descrição do teste:	O médico tenta acessar o painel regional.
Dados de entrada:	Token de médico.
Resultado:	Sistema nega (código 403).
Resultado obtido:	Código 403. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 45 – Caso de teste CT44 – Gerar painel regional, teste 4

Nome:	Painel no site
Pré-condição:	Site ligado à API.
Descrição do teste:	O gestor entra no site e abre o painel.
Dados de entrada:	Credenciais de gestor do seed.
Resultado:	Site mostra indicadores e gráficos por região.
Resultado obtido:	214.527 atendimentos em 6 meses e gráficos por região. (manual; passou)

Fonte: Elaborado pelos autores (2026).

2.12 Processo: Proteger e publicar a API na nuvem

Proteger e Publicar a API reúne os requisitos não funcionais de segurança e de nuvem. Toda resposta da API leva cabeçalhos de segurança e só é liberada pelo navegador para as origens autorizadas (CORS). O usuário acessa a API pelo CloudFront, sempre com HTTPS; o balanceador de carga só aceita requisições que venham do CloudFront, e um acesso direto recebe <acesso negado>. Após cada implantação, um teste de fumaça confere as funções principais.

Plano de testes

Teste1: cabeçalhos de segurança

Entrada1: qualquer requisição

Saída1: HSTS, nosniff, CSP, sem X-Powered-By

Teste2: origem não autorizada (CORS)

Entrada2: requisição de outro site

Saída2: navegador bloqueia a resposta

Teste3: acesso direto ao balanceador

Entrada3: http://<balanceador>/api/health

Saída3: acesso negado

Teste4: HTTP redirecionado para HTTPS

Entrada4: http://<cloudfront>/api/health

Saída4: redirecionamento para HTTPS

Teste5: teste de fumaça na AWS

Entrada5: script smoke-test.mjs

Saída5: 9 verificações aprovadas

Casos de teste

Quadro 46 – Caso de teste CT45 – Proteger e publicar a API na nuvem, teste 1

Nome:	Cabeçalhos de segurança
Pré-condição:	API em execução.
Descrição do teste:	Faz-se uma requisição e examinam-se os cabeçalhos.
Dados de entrada:	Requisição comum.
Resultado:	Cabeçalhos de segurança presentes e tecnologia do servidor oculta.
Resultado obtido:	Presentes, também na AWS. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 47 – Caso de teste CT46 – Proteger e publicar a API na nuvem, teste 2

Nome:	Origem não autorizada (CORS)
Pré-condição:	API em execução.
Descrição do teste:	Simula-se uma chamada vinda de um site não autorizado.
Dados de entrada:	Origem desconhecida.
Resultado:	Resposta sem o cabeçalho de liberação para essa origem.
Resultado obtido:	Origem bloqueada. (automatizado; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 48 – Caso de teste CT47 – Proteger e publicar a API na nuvem, teste 3

Nome:	Acesso direto ao balanceador
Pré-condição:	API implantada na AWS.
Descrição do teste:	Acessa-se o balanceador sem passar pelo CloudFront.
Dados de entrada:	Endereço do balanceador.
Resultado:	Balanceador recusa (código 403).
Resultado obtido:	Código 403. (manual; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 49 – Caso de teste CT48 – Proteger e publicar a API na nuvem, teste 4

Nome:	HTTP redirecionado para HTTPS
Pré-condição:	API implantada na AWS.
Descrição do teste:	Acessa-se a API por HTTP.
Dados de entrada:	Endereço sem HTTPS.
Resultado:	CloudFront redireciona (código 301).
Resultado obtido:	Código 301. (manual; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 50 – Caso de teste CT49 – Proteger e publicar a API na nuvem, teste 5

Nome:	Teste de fumaça na AWS
Pré-condição:	API implantada e banco com o seed.
Descrição do teste:	Executa-se o teste de fumaça contra a API publicada.
Dados de entrada:	Endereço da API na AWS.
Resultado:	As nove verificações passam.
Resultado obtido:	9 de 9. (manual; passou)

Fonte: Elaborado pelos autores (2026).

2.13 Processo: Escalar a API sob carga

Escalar a API garante que o sistema continue respondendo quando o número de usuários cresce. O balanceador de carga distribui as requisições entre as tarefas saudáveis. A cada minuto, a CPU média é medida: se passa de 50%, uma tarefa é criada, até o limite de três; se fica abaixo de 45% por 15 minutos, uma tarefa é removida. Os detalhes e as medições estão na análise de escalabilidade e custo.

Plano de testes

Teste1: escalonamento automático sob carga

Entrada1: 20 usuários fazendo login por 10 minutos

Saída1: sobe para 2 ou mais tarefas e volta para 1

Teste2: taxa de erro sob carga

Entrada2: mesma carga

Saída2: taxa de erro menor que 1%

Teste3: tempo de resposta com três tarefas

Entrada3: mesma carga

Saída3: tempo menor que com uma tarefa

Casos de teste

Quadro 51 – Caso de teste CT50 – Escalar a API sob carga, teste 1

Nome:	Escalonamento automático sob carga
Pré-condição:	API implantada na AWS; limite de login desligado.
Descrição do teste:	Executa-se o cenário C03 e acompanha-se o número de tarefas.
Dados de entrada:	Carga contínua de logins.
Resultado:	O serviço aumenta as tarefas na carga e reduz sem carga.
Resultado obtido:	1 → 2 → 3 tarefas e volta para 1 cerca de 20 minutos depois. (manual; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 52 – Caso de teste CT51 – Escalar a API sob carga, teste 2

Nome:	Taxa de erro sob carga
Pré-condição:	API implantada na AWS.
Descrição do teste:	Mede-se a proporção de requisições com erro.
Dados de entrada:	Carga contínua de logins.
Resultado:	Menos de 1% de erros.
Resultado obtido:	Antes da correção: 1,1% a 3,1% (erro 502). Depois: 0% em 3.659 requisições. (manual; passou)

Fonte: Elaborado pelos autores (2026).

Quadro 53 – Caso de teste CT52 – Escalar a API sob carga, teste 3

Nome:	Tempo de resposta com três tarefas
Pré-condição:	API implantada na AWS.
Descrição do teste:	Compara-se o tempo de resposta do C03 com o do C02.
Dados de entrada:	Carga contínua de logins.
Resultado:	Tempo de resposta menor com mais tarefas.
Resultado obtido:	Mediana caiu de 7,3 s para cerca de 1 s, mas os 5% mais lentos ficaram perto de 8,5 s. (manual; passou com ressalva)

Fonte: Elaborado pelos autores (2026).

2.14 Resultados da execução dos testes

A Tabela 1 resume a execução por processo. Dos 52 casos, 51 passaram e 1 passou com ressalva. A suíte automatizada da API reúne 231 testes, todos aprovados em 9 de novembro de 2026, com 96,9% das linhas de código cobertas.

Tabela 1 – Resultado da execução por processo

Processo	Casos	Automatizados	Manuais	Passaram	Com ressalva
Autenticar profissional	5	4	1	5	0
Autenticar paciente	4	4	0	4	0
Consultar prontuário unificado	5	4	1	5	0
Cadastrar paciente	2	2	0	2	0
Registrar atendimento	7	6	1	7	0
Anexar e baixar documento	5	4	1	5	0
Verificar autenticidade de documento	2	2	0	2	0
Controlar privacidade	5	5	0	5	0
Consultar dados no aplicativo	5	5	0	5	0
Gerar painel regional	4	3	1	4	0
Proteger e publicar a API na nuvem	5	2	3	5	0
Escalar a API sob carga	3	0	3	2	1
Total	**52**	**41**	**11**	**51**	**1**

Fonte: Elaborado pelos autores (2026), com dados da execução dos testes.

Durante a execução, foram encontrados os defeitos do Quadro 54. Três foram corrigidos e confirmados por nova execução; o quarto, que afeta apenas o tempo de resposta sob carga, permanece como melhoria proposta.

Quadro 54 – Defeitos encontrados nos testes

Código	Defeito	Correção	Situação
D1	Documentação da API (Swagger) desatualizada depois de mudar uma rota	Arquivo regenerado com o comando de geração	Corrigido
D2	Erros 502 sob carga na AWS	API mantém a conexão por 65 s, mais que os 60 s do balanceador	Corrigido
D3	Testes de atendimento falhando entre 21h e meia-noite	Teste passou a usar a data de Brasília, igual à API	Corrigido
D4	Carga desigual entre as tarefas com três instâncias	Proposta: algoritmo *least outstanding requests* no balanceador	Em aberto

Fonte: Elaborado pelos autores (2026).

3 Arquitetura e funcionamento do balanceamento

3.1 Arquitetura implantada na AWS

A API foi implantada na região da Virgínia do Norte (us-east-1), escolhida por ser a mais barata e por ser uma das liberadas para a conta utilizada no projeto. Toda a infraestrutura é descrita como código, com o AWS CDK, o que permite criá-la e removê-la por comandos, sem configuração manual. O Quadro 55 resume os componentes.

Quadro 55 – Componentes da infraestrutura na nuvem

Componente	Função no sistema
Amazon CloudFront	Recebe as requisições dos usuários com HTTPS e as encaminha ao balanceador de carga
Application Load Balancer (ALB)	Distribui as requisições entre as instâncias da API e só aceita tráfego vindo do CloudFront
Amazon ECS com Fargate Spot	Executa a API em contêineres (tarefas) de 0,25 vCPU e 512 MB, de 1 a 3 tarefas
Auto scaling do ECS	Aumenta ou reduz o número de tarefas conforme o uso médio de CPU
MongoDB Atlas	Banco de dados dos prontuários, atendimentos e auditoria
Amazon S3	Armazenamento das receitas, laudos e exames em PDF
Amazon CloudWatch e SNS	Métricas, painel de monitoramento, alarmes e avisos por e-mail

Fonte: Elaborado pelos autores (2026).

3.2 Balanceamento de carga e escalonamento automático

O balanceador de carga recebe todas as requisições e as distribui entre as tarefas saudáveis, isto é, aquelas que respondem à verificação de saúde em */api/health/live*. O escalonamento automático observa a CPU média do serviço a cada minuto: quando ela passa de 50%, uma nova tarefa é criada, até o limite de três; quando permanece abaixo de 45% por 15 minutos, uma tarefa é removida. Uma tarefa nova leva cerca de um minuto e meio para começar a receber requisições.

A demora intencional para reduzir o número de tarefas evita que o sistema desligue instâncias em uma pausa curta da demanda e precise criá-las de novo logo em seguida.

4 Procedimentos metodológicos

4.1 Testes de carga

Os testes utilizaram um script próprio, escrito em Node.js, que simula usuários simultâneos fazendo login sem parar. O login foi escolhido por ser a operação mais pesada da API: a verificação da senha com o algoritmo bcrypt consome CPU de propósito, para dificultar ataques às senhas. Isso faz a CPU subir rapidamente e permite observar o escalonamento. Durante os testes, o limite de tentativas de login foi desligado e religado ao final. A Tabela 2 descreve os cenários.

Tabela 2 – Cenários dos testes de carga na AWS

Cenário	Configuração	Usuários simultâneos	Duração	Objetivo
C02	1 tarefa fixa	20	5 min	Capacidade de uma tarefa
C03	Escalonamento de 1 a 3 tarefas	20	10 min	Verificar o aumento de tarefas
C04	Sem carga, após o C03	0	~20 min	Verificar a redução para 1 tarefa

Fonte: Elaborado pelos autores (2026).

Foram realizadas três rodadas. A primeira revelou erros de conexão; após a correção (seção 5.2), a segunda rodada repetiu os cenários. A terceira rodada foi executada a partir de outra cópia do projeto, ainda sem a correção, o que permitiu confirmar a causa do problema. As métricas de CPU, tarefas saudáveis, requisições e tempo de resposta foram exportadas do Amazon CloudWatch.

4.2 Medição da capacidade por operação

Para estimar o custo de uso real, que mistura logins e consultas, mediu-se a capacidade de um processo da API em computador local (AMD Ryzen 5 5600GT) para cinco operações, com dez usuários simultâneos. A proporção entre o resultado local e o medido na AWS para o login foi usada para estimar a capacidade de uma tarefa na nuvem nas demais operações.

4.3 Estimativa de custos

Os custos foram calculados a partir dos preços públicos de referência da AWS e do MongoDB Atlas, sob demanda, convertidos com o dólar a R$ 5,50. Os valores são aproximados e devem ser conferidos na calculadora oficial antes de qualquer uso formal (Amazon Web Services, 2026a; MongoDB, 2026). As premissas de uso foram definidas pela equipe, e o número de atendimentos diários na rede hospitalar brasileira (1,5 a 2,5 milhões) é uma estimativa adotada pela equipe, e não um dado oficial.

5 Resultados e discussão

5.1 Testes de carga na AWS

A Tabela 3 apresenta os resultados das três rodadas. Em todas, o escalonamento automático aumentou o serviço de uma para três tarefas durante a carga e voltou a uma tarefa depois dela.

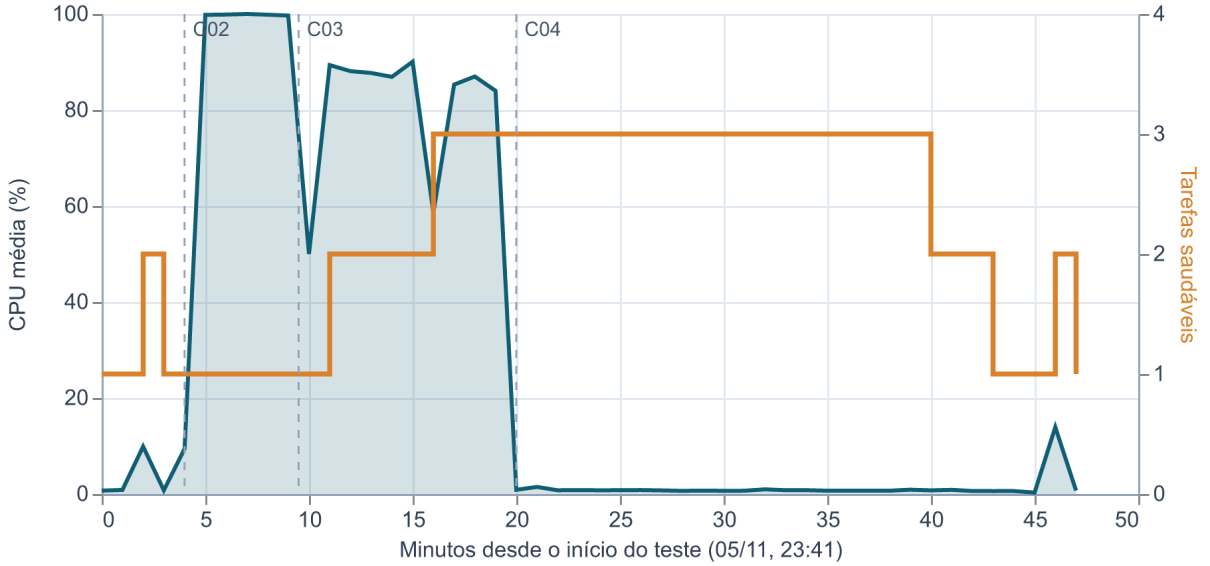
Tabela 3 – Resultados dos testes de carga por rodada

Rodada	Versão da API	C02: vazão	C02: erros	C03: vazão	C03: erros	Tarefas
1 (05/11, 22h58)	Sem a correção	2,8 req/s	1,65%	6,0 req/s	1,09%	1 → 3 → 1
2 (05/11, 23h45)	Com a correção	2,8 req/s	0%	4,7 req/s	0%	1 → 3 → 1
3 (06/11, 01h13)	Sem a correção	2,9 req/s	3,14%	5,9 req/s	1,21%	1 → 3 → 1

Fonte: Elaborado pelos autores (2026), com dados dos testes de carga e do Amazon CloudWatch.

O Gráfico 1 mostra, minuto a minuto, a CPU média do serviço e o número de tarefas saudáveis durante a rodada 2. Com uma tarefa (C02), a CPU permanece em 100%. Ao liberar o escalonamento (C03), a CPU passa de 50% e o serviço sobe para duas e depois três tarefas. Sem carga (C04), a CPU cai para perto de zero e, cerca de 20 minutos depois, o serviço volta a uma tarefa, como detalha a Tabela 4.

Gráfico 1 – CPU média e tarefas saudáveis durante a rodada 2



Fonte: Elaborado pelos autores (2026), com dados do Amazon CloudWatch.

Tabela 4 – Linha do tempo do escalonamento automático (horário de Brasília)

Rodada	Subiu para 2	Subiu para 3	Fim da carga	Desceu para 2	Voltou para 1
1	23h04	23h10	23h13	23h31	23h34
2	23h51	23h57	00h00	00h21	00h24

Fonte: Elaborado pelos autores (2026), com dados do histórico do *auto scaling* da AWS.

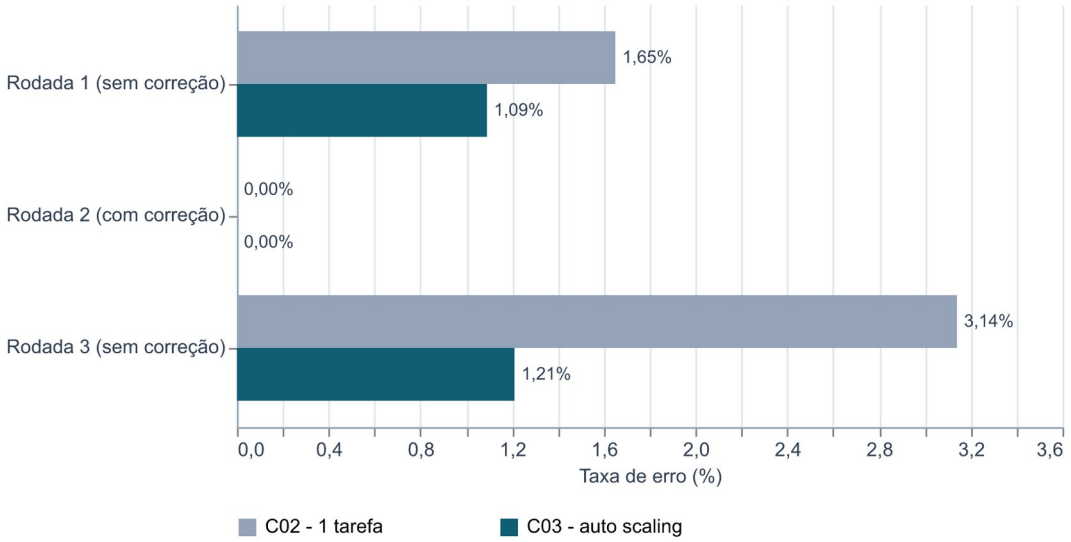
Na rodada 1, com três tarefas ao final da carga, a vazão chegou a 8,4 logins por segundo, o triplo da obtida com uma tarefa (2,8 por segundo). O crescimento proporcional ao número de tarefas indica que a aplicação escala horizontalmente. O alarme de lentidão (tempo de resposta acima de 2 segundos) também disparou durante a carga e avisou por e-mail quando a situação se normalizou, o que confirma o funcionamento do monitoramento.

5.2 Defeito de conexão encontrado e corrigido

Nas rodadas sem a correção, a taxa de erro ficou entre 1,09% e 3,14%, acima do critério de aceitação de 1%, conforme o Gráfico 2. Todos os erros eram respostas 502 geradas pelo próprio balanceador de carga; a API não registrou nenhuma falha. A causa é a diferença entre os tempos de conexão ociosa: o Node.js encerra uma conexão parada após 5 segundos, enquanto o balanceador a reaproveita por até 60 segundos. Quando o balanceador envia uma requisição por uma conexão que acabou de ser encerrada, devolve erro ao usuário.

A correção consistiu em fazer a API manter a conexão por 65 segundos, mais do que o balanceador. Na rodada 2, com a correção, foram 3.659 requisições sem nenhum erro. A rodada 3, executada com uma versão sem a correção, voltou a apresentar erros, o que confirma a causa.

Gráfico 2 – Taxa de erro por rodada e cenário



Fonte: Elaborado pelos autores (2026), com dados dos testes de carga.

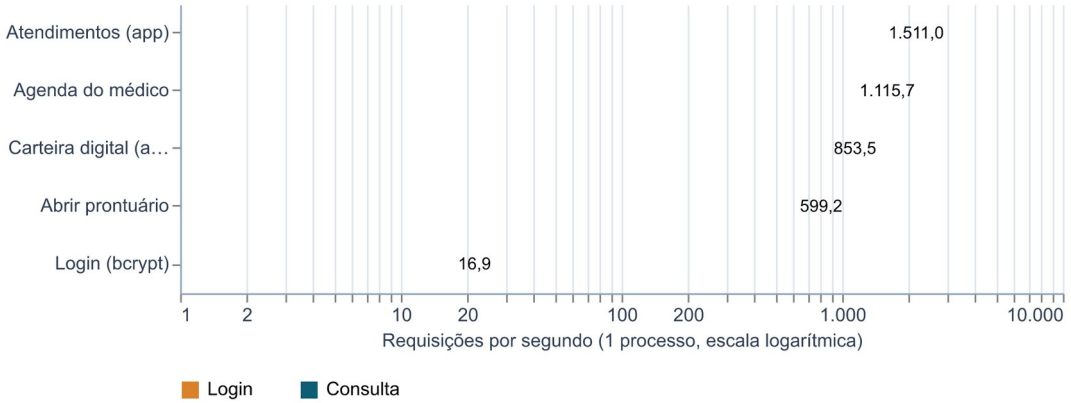
5.3 Distribuição desigual da carga

Na rodada 2, com três tarefas, o tempo de resposta típico (mediana) caiu de 7,3 segundos para cerca de 1 segundo, mas os 5% de requisições mais lentas continuaram perto de 8,5 segundos, e a CPU média ficou em 87%, e não em 100%. Isso indica que uma das tarefas recebeu mais trabalho que as demais. O balanceador distribui por quantidade de requisições (*round robin*), mas um login demora muito mais que outras operações. Como melhoria, propõe-se trocar o algoritmo para *least outstanding requests*, que envia cada requisição à tarefa com menos trabalho pendente, e repetir o cenário C03.

5.4 Capacidade por operação

O Gráfico 3 e a Tabela 5 mostram que as consultas são de 35 a 90 vezes mais leves que o login. Para as estimativas de custo, considerou-se uma mistura de 5% de logins e 95% de consultas, todas tão pesadas quanto abrir o prontuário (cenário pessimista). Nessas condições, uma tarefa de 0,25 vCPU atende cerca de 36 requisições por segundo com a CPU em 100%, ou 18 requisições por segundo com o alvo de 50% do escalonamento automático.

Gráfico 3 – Capacidade de um processo da API por operação



Fonte: Elaborado pelos autores (2026), com dados de medição local.

Tabela 5 – Capacidade medida em computador local e estimada para uma tarefa na AWS

Operação	Local (1 processo)	AWS (0,25 vCPU)
Login (bcrypt)	16,9 req/s	2,8 req/s (medido)
Abrir prontuário (com auditoria)	599 req/s	~100 req/s
Agenda do médico	1.116 req/s	~185 req/s
Carteira digital (aplicativo)	854 req/s	~140 req/s
Atendimentos do paciente (aplicativo)	1.511 req/s	~250 req/s

Fonte: Elaborado pelos autores (2026).

Nota: os valores da AWS, exceto o login, aplicam a proporção medida no login (2,8 ÷ 16,9 = 0,17).

5.5 Custo conforme o porte

Para estimar o custo por porte, adotaram-se as premissas do Quadro 56. O pico de requisições por segundo foi calculado como o número de pacientes multiplicado por 20 requisições e dividido pelos segundos de 30 dias de 12 horas, multiplicado pelo fator de pico.

Quadro 56 – Premissas de uso para a estimativa por porte

Premissa	Valor adotado	Justificativa
Requisições por paciente por mês	20	Abrir o aplicativo duas vezes por mês e as consultas do médico no atendimento
Horário de uso	12 horas por dia	Funcionamento das unidades de saúde e uso diurno do aplicativo
Pico	3 vezes a média	Manhãs de segunda-feira e campanhas de vacinação
Documentos	2 por paciente por ano, 300 KB cada	Receitas e laudos em PDF
Câmbio	R$ 5,50 por dólar	—

Fonte: Elaborado pelos autores (2026).

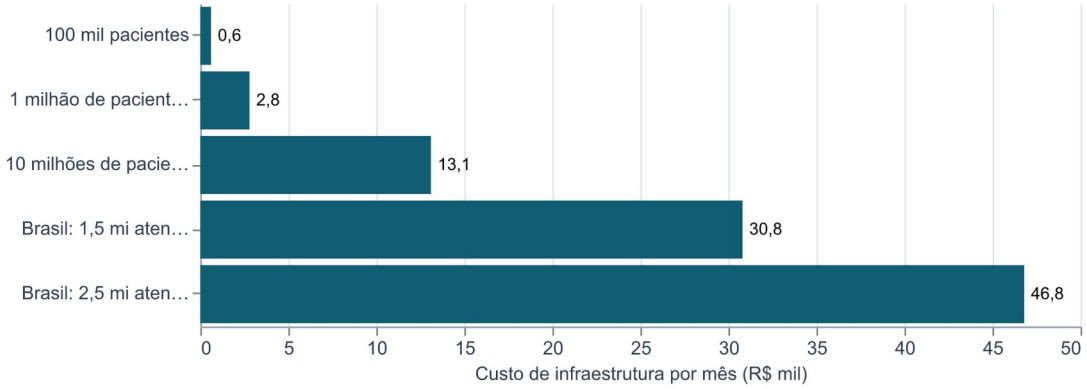
Tabela 6 – Custo mensal estimado da infraestrutura por porte (US$)

Item	100 mil pacientes	1 milhão	10 milhões
Tarefas da API (Fargate, sob demanda)	18	27	252
Balanceador de carga	17	18	24
Endereços IP públicos	15	18	33
CloudFront	0	22	418
MongoDB Atlas	58	394	1.460
Amazon S3 (documentos)	2	16	160
CloudWatch	1	3	15
Redis (limite de login entre tarefas)	—	12	23
Total por mês	**110 (R$ 600)**	**510 (R$ 2.800)**	**2.390 (R$ 13.100)**
Por 1 milhão de requisições	R$ 300	R$ 140	R$ 65

Fonte: Elaborado pelos autores (2026), com base nos preços de referência da AWS e do MongoDB Atlas.

Observa-se que, quanto maior o porte, menor o custo por paciente, pois a parte fixa (balanceador e banco de dados) se divide entre mais usuários. No porte de 1 milhão de pacientes, o banco de dados representa cerca de 77% do custo, e o maior volume nele é a trilha de auditoria. O Gráfico 4 compara os portes, incluindo o cenário de abrangência nacional, detalhado na seção seguinte.

Gráfico 4 – Custo mensal de infraestrutura por porte



Fonte: Elaborado pelos autores (2026).

5.6 Cenário de abrangência nacional

Considerou-se o caso extremo de toda a rede hospitalar brasileira usar a plataforma, com 1,5 a 2,5 milhões de atendimentos por dia, incluindo pronto-socorro, exames e internações. A Tabela 7 apresenta quantas requisições cada atendimento gera.

Tabela 7 – Requisições geradas por atendimento

Ação	Requisições
Médico abre o prontuário	1
Médico consulta o histórico do paciente	1
Atualização da agenda do médico	1
Registro do atendimento	1
Envio de receita ou laudo	1
Download ou conferência de um documento	1
Paciente consulta o aplicativo depois	2
Logins (médico a cada 20 pacientes; metade dos pacientes no aplicativo)	0,55
Total por atendimento	**8,55**

Fonte: Elaborado pelos autores (2026).

Com 70% do movimento entre 7h e 19h e um pico igual ao dobro da média desse horário, obtêm-se os volumes da Tabela 8. A capacidade usada foi de cerca de 62 requisições por segundo por vCPU, com o escalonamento em 50%.

Tabela 8 – Volume e capacidade necessários no cenário nacional

Indicador	1,5 milhão/dia	2,5 milhões/dia
Requisições por dia	12,8 milhões	21,4 milhões
Requisições por mês	385 milhões	641 milhões
Pico	416 req/s	693 req/s
vCPUs no pico	6,7 (7 tarefas de 1 vCPU)	11,2 (12 tarefas de 1 vCPU)
Tráfego de saída por mês	~1,9 TB	~3,2 TB
Documentos novos por mês	~6,8 TB	~11 TB
Crescimento do banco por ano	~1,2 TB	~2 TB

Fonte: Elaborado pelos autores (2026).

Tabela 9 – Custo mensal estimado no cenário nacional (US$)

Item	1,5 milhão/dia	2,5 milhões/dia
Tarefas da API (Fargate)	104	157
Balanceador de carga	32	42
Endereços IP públicos	22	29
CloudFront	926	1.632
MongoDB Atlas	~3.200	~4.500
Amazon S3 (documentos, ao fim do 1º ano)	~1.000	~1.670
CloudWatch	25	35
Redis	47	47
WAF (firewall de aplicação)	240	395
Total por mês	**~5.600 (R$ 30.800)**	**~8.500 (R$ 46.800)**
Custo por atendimento	R$ 0,0007	R$ 0,0006

Fonte: Elaborado pelos autores (2026), com base nos preços de referência da AWS e do MongoDB Atlas.

Os servidores da API representam apenas cerca de 2% do custo: de 7 a 12 tarefas de 1 vCPU atendem o pico do país, bastando elevar o limite do escalonamento automático. O custo está concentrado nos dados, com o banco respondendo por 53% a 57% do total e os documentos por cerca de 20%. Mesmo nesse cenário, o custo por atendimento fica abaixo de um décimo de centavo.

5.7 Banco de dados e arquivos ao longo de dez anos

5.7.1 O que a legislação permite descartar

Antes de propor o descarte de dados antigos, é necessário verificar os prazos legais. A Lei nº 13.787/2018 estabelece que os prontuários podem ser eliminados somente após o prazo mínimo de 20 anos a partir do último registro (Brasil, 2018b), e parte das interpretações da Resolução CFM nº 1.821/2007 aponta guarda permanente para o prontuário nativamente eletrônico (Conselho Federal de Medicina, 2007; 2026). O Quadro 57 resume a situação de cada tipo de dado.

Quadro 57 – Possibilidade de descarte por tipo de dado

Dado	Pode ser descartado após 5 anos?	Fundamento
Prontuário (documento do paciente e histórico de atendimentos)	Não	Mínimo de 20 anos após o último registro (Brasil, 2018b); possível guarda permanente para o eletrônico (Conselho Federal de Medicina, 2007)
Receitas, laudos e exames anexados	Não	Fazem parte do prontuário
Trilha de auditoria (registro de acessos)	Sim, por política adotada	Não é prontuário; o Marco Civil da Internet exige ao menos 6 meses (Brasil, 2014). Adotou-se guarda de 5 anos

Fonte: Elaborado pelos autores (2026).

Assim, em dez anos nenhum dado do prontuário é descartado. A economia vem de mover os dados antigos para armazenamento de baixo custo (S3 Glacier), e não de apagá-los.

5.7.2 Política de armazenamento por idade do dado

O modelo da API mantém um documento por paciente, atualizado a cada atendimento pelo médico responsável, e registros separados para o histórico de

atendimentos e para a auditoria. Os arquivos ficam no S3. O Quadro 58 apresenta onde cada dado fica conforme sua idade.

Quadro 58 – Local de armazenamento conforme a idade do dado

Idade	Documento do paciente	Histórico de atendimentos	Auditoria	PDFs
Até 90 dias	Atlas	Atlas	Atlas	S3 Standard
90 dias a 2,5 anos	Atlas	Atlas	Atlas	S3 Standard-IA
Mais de 2,5 anos	Atlas (sempre o atual)	S3 Glacier Instant Retrieval	S3 Glacier Deep Archive	S3 Glacier Instant Retrieval
Mais de 5 anos	Atlas	Glacier	Descartada	Glacier

Fonte: Elaborado pelos autores (2026).

O Glacier Instant Retrieval devolve o arquivo em milissegundos, de modo que o médico continua abrindo um exame antigo normalmente, com armazenamento cerca de seis vezes mais barato que o S3 comum (Amazon Web Services, 2026c).

5.7.3 Resultados da simulação

Quadro 59 – Premissas da simulação de dez anos

Premissa	Valor adotado
Pacientes cadastrados	80 milhões no 1º ano, 150 milhões no 5º e 190 milhões no 10º
Documento do paciente	8 KB, crescendo até 10 KB em dez anos
Cada atendimento	1,5 KB de histórico e 0,75 KB de auditoria
PDFs	Metade dos atendimentos gera um arquivo de 300 KB
Compactação ao arquivar	O JSON arquivado ocupa 30% do tamanho original
Custo do Atlas	US$ 1,50 por GB por mês, incluindo três cópias, backup e servidores
Preços do S3 (por GB por mês)	Standard US$ 0,023; Standard-IA 0,0125; Glacier Instant Retrieval 0,004; Glacier Deep Archive 0,00099

Fonte: Elaborado pelos autores (2026).

Tabela 10 – Evolução anual no cenário de 1,5 milhão de atendimentos por dia

Ano	Pacientes	Dados no Atlas	Histórico no Glacier	PDFs	Custo no mês (US$)	Sem arquivar (US$)
1	80 mi	1,9 TB	—	82 TB	4.074	4.074
3	120 mi	4,1 TB	0,2 TB	246 TB	9.115	10.388
5	150 mi	4,4 TB	0,9 TB	411 TB	10.251	16.614
7	166 mi	4,6 TB	1,4 TB	575 TB	11.226	22.679
10	190 mi	5,0 TB	2,2 TB	821 TB	12.724	31.813

Fonte: Elaborado pelos autores (2026).

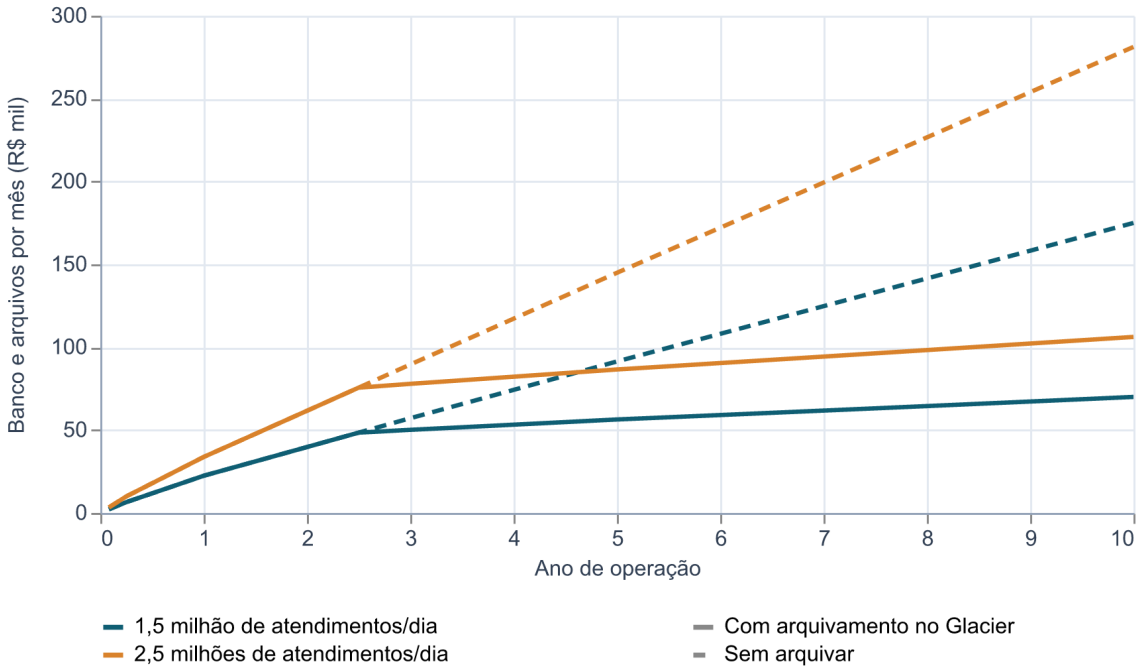
Tabela 11 – Evolução anual no cenário de 2,5 milhões de atendimentos por dia

Ano	Pacientes	Dados no Atlas	Histórico no Glacier	PDFs	Custo no mês (US$)	Sem arquivar (US$)
1	80 mi	2,7 TB	—	137 TB	6.134	6.134
3	120 mi	6,2 TB	0,3 TB	411 TB	14.160	16.281
5	150 mi	6,5 TB	1,5 TB	684 TB	15.736	26.340
7	166 mi	6,7 TB	2,4 TB	958 TB	17.150	36.238
10	190 mi	7,0 TB	3,6 TB	1.369 TB	19.307	51.121

Fonte: Elaborado pelos autores (2026).

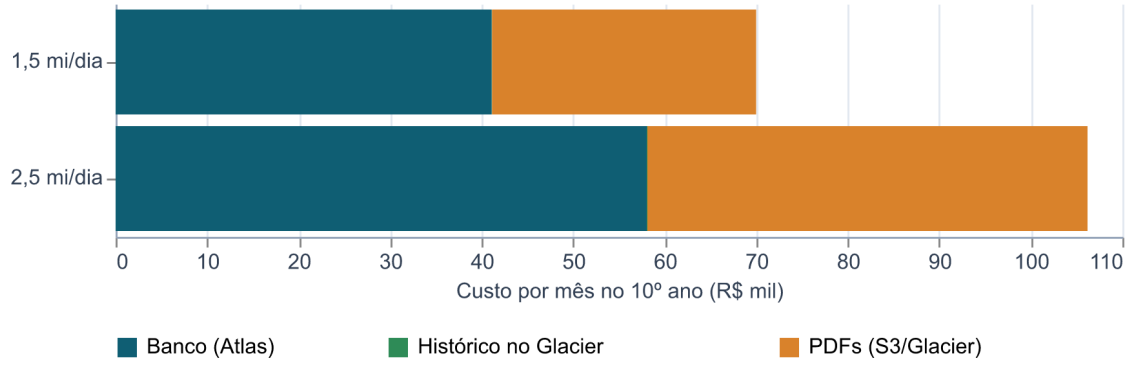
O Gráfico 5 mostra que, com o arquivamento, o custo mensal deixa de crescer na mesma velocidade a partir de 2,5 anos, pois o banco se estabiliza entre 5 e 7 TB. Sem arquivamento, o banco chegaria a 14 a 22 TB e o custo seria cerca de três vezes maior. O Gráfico 6 mostra que, no décimo ano, os PDFs passam a representar de 40% a 45% do custo.

Gráfico 5 – Custo mensal de banco e arquivos ao longo de dez anos



Fonte: Elaborado pelos autores (2026).

Gráfico 6 – Composição do custo mensal no décimo ano, com arquivamento



Fonte: Elaborado pelos autores (2026).

Tabela 12 – Custo total de banco e arquivos em dez anos

Política	1,5 milhão/dia	2,5 milhões/dia
Com arquivamento no Glacier	US$ 1,12 milhão (~R$ 6,2 milhões)	US$ 1,72 milhão (~R$ 9,5 milhões)
Sem arquivamento	US$ 1,99 milhão (~R$ 10,9 milhões)	US$ 3,17 milhões (~R$ 17,4 milhões)
Economia	44%	46%

Fonte: Elaborado pelos autores (2026).

Para reduzir ainda mais o custo, os PDFs com mais de cinco anos podem ir para o Glacier Deep Archive, quatro vezes mais barato, com recuperação em até 12 horas, o que economizaria cerca de US$ 1.200 por mês no décimo ano do cenário menor. Comprimir as imagens dos exames digitalizados também reduziria o volume.

5.8 Infraestrutura própria em comparação com a nuvem

Avaliou-se a alternativa de construir infraestrutura dedicada, separada em uma estrutura para os dados e outra para a API e o site, cada uma em dois *data centers* (principal e contingência), como exige um sistema nacional de saúde. O Quadro 60 descreve os componentes.

Quadro 60 – Componentes de uma infraestrutura própria

Estrutura	Componentes em cada data center	Equivalente na AWS
Dados	Três servidores de banco com SSD NVMe; armazenamento de objetos próprio (Ceph ou MinIO) com cerca de 1 a 1,7 PB úteis em dez anos; biblioteca de fitas LTO na contingência	MongoDB Atlas, S3 e Glacier
Aplicação	Três servidores para API e site, par de firewalls e balanceadores, switches, dois links de internet e proteção contra DDoS	ECS Fargate, ALB, CloudFront e WAF

Fonte: Elaborado pelos autores (2026).

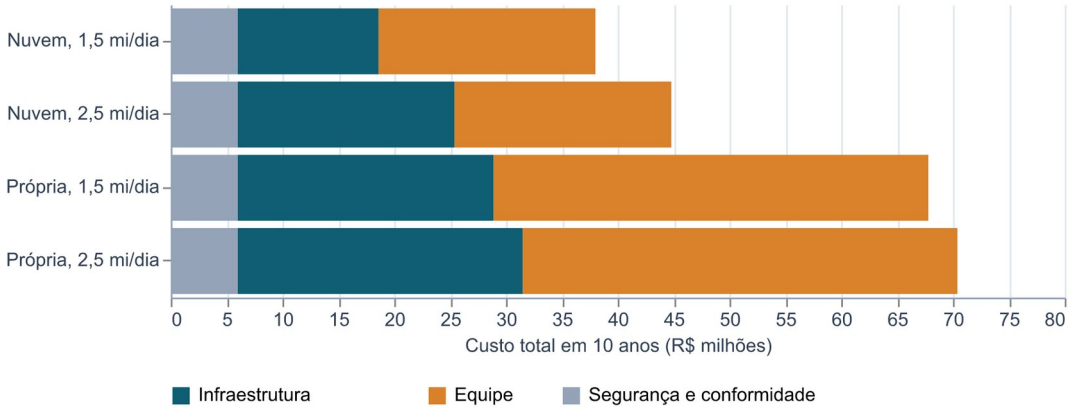
Para uma comparação justa, ambas as opções incluem contingência, dados no Brasil, suporte do fornecedor, equipe de operação e segurança. A Tabela 13 e o Gráfico 7 apresentam o resultado.

Tabela 13 – Custo total em dez anos: nuvem e infraestrutura própria (R$ milhões)

Item	Nuvem 1,5 mi/dia	Nuvem 2,5 mi/dia	Própria 1,5 mi/dia	Própria 2,5 mi/dia
Banco e arquivos	6,2	9,5	—	—
API, site, CDN e firewall	0,9	1,5	—	—
Contingência em outra região	2,2	3,3	incluída	incluída
Região São Paulo (~35% mais cara)	2,5	3,9	—	—
Suporte do fornecedor (~7%)	0,8	1,3	—	—
Equipamentos (compra e troca no 5º ano)	—	—	4,9	5,6
Manutenção dos equipamentos	—	—	2,4	2,8
Espaço em data center (R$ 12 mil por rack/mês)	—	—	7,2	8,6
Links de internet e enlace entre sites	—	—	6,6	6,6
Proteção contra DDoS	—	—	1,8	1,8
Subtotal de infraestrutura	**12,6**	**19,4**	**22,9**	**25,5**
Equipe de operação (R$ 18 mil + encargos)	19,4	19,4	38,9	38,9
Segurança e conformidade	6,0	6,0	6,0	6,0
Total em dez anos	**38**	**45**	**68**	**70**
Média por mês (R$ mil)	317	374	565	586

Fonte: Elaborado pelos autores (2026).

Gráfico 7 – Custo total em dez anos por tipo de infraestrutura



Fonte: Elaborado pelos autores (2026).

O equipamento é a menor parte do custo da infraestrutura própria, cerca de R$ 5 milhões em dez anos. O que pesa é o espaço com energia e refrigeração, os links redundantes, os contratos de manutenção e, principalmente, a equipe de plantão 24

horas, estimada em dez pessoas, contra cinco na nuvem, onde parte do trabalho é feita pelo fornecedor.

Tabela 14 – Arquivos em infraestrutura própria e na nuvem em dez anos (modelo híbrido)

Opção	1,5 milhão/dia	2,5 milhões/dia
PDFs na nuvem (região São Paulo)	~R$ 3,0 milhões	~R$ 5,1 milhões
PDFs em armazenamento próprio, com fitas, espaço, enlace e três pessoas de plantão	~R$ 20,6 milhões	~R$ 21,2 milhões

Fonte: Elaborado pelos autores (2026).

Manter apenas os arquivos em infraestrutura própria também não compensa nesse volume, como mostra a Tabela 14. A infraestrutura própria se torna competitiva quando o *data center* e a equipe já existem, como ocorre em órgãos públicos como DATASUS, Serpro e Dataprev: sem aluguel de espaço e com equipe compartilhada, o custo cai para cerca de R$ 33 a 35 milhões em dez anos, próximo ao da nuvem. Também compensa em escalas muito maiores ou quando há exigência de soberania de dados que impeça a nuvem pública. A arquitetura adotada permite essa migração sem reescrever a API, pois MongoDB e armazenamentos compatíveis com S3 funcionam em qualquer ambiente.

5.9 Realismo da estimativa

As estimativas das seções 5.5 a 5.7 consideram apenas a infraestrutura, com preços de referência na região da Virgínia. Essa parte está na ordem de grandeza correta, mas o custo de operar um sistema desse tamanho é de quatro a cinco vezes maior, como mostra a Tabela 15.

Tabela 15 – Infraestrutura isolada e operação realista na nuvem

Indicador	Somente infraestrutura (Virgínia)	Operação realista
Total em dez anos	R$ 7 a 11 milhões	R$ 38 a 45 milhões
Média por mês	R$ 59 a 92 mil	R$ 317 a 374 mil
Custo por atendimento	~R$ 0,0013	~R$ 0,005 a 0,007

Fonte: Elaborado pelos autores (2026).

A operação realista acrescenta a região de São Paulo, a contingência, o suporte do fornecedor, a equipe de operação, que representa mais da metade do custo, e a segurança. Ainda ficam fora da estimativa itens que, em um projeto real, podem superar todos os anteriores:

a) equipe de desenvolvimento contínuo do sistema;
b) integração com os sistemas de cada hospital, nos padrões HL7 e FHIR;
c) central de suporte a médicos e pacientes, e treinamento;
d) assessoria jurídica e encarregado de dados, conforme a LGPD (Brasil, 2018a);
e) certificados digitais ICP-Brasil para assinatura das receitas.

6 Considerações finais

O plano de testes descreveu 12 processos do sistema e os verificou por meio de 52 casos de teste, dos quais 51 passaram e 1 passou com ressalva. A descrição de cada processo, com as entradas e as saídas esperadas, orientou a definição dos testes e permitiu encontrar e corrigir três defeitos, entre eles erros de conexão que só apareciam com a API publicada na nuvem e sob carga.

Os testes na AWS mostraram que o balanceamento de carga e o escalonamento automático funcionam: a API aumentou de uma para três tarefas sob carga, voltou a uma tarefa sem carga e teve vazão proporcional ao número de tarefas. Permanece como melhoria a distribuição desigual da carga entre as tarefas, para a qual se propõe a troca do algoritmo de balanceamento.

Quanto ao custo, a infraestrutura em nuvem para o país inteiro custaria de R$ 30 a 47 mil por mês, e a operação realista, com equipe, contingência e segurança, de R$ 317 a 374 mil por mês, sempre abaixo de um centavo por atendimento. O maior custo está nos dados, e não nos servidores da API. Em dez anos, mover para o Glacier os dados com mais de 2,5 anos reduz em 44% a 46% o custo de banco e arquivos, sem descartar nenhum dado do prontuário, que deve ser guardado por pelo menos 20 anos. Para o porte do projeto, a nuvem se mostrou mais barata que uma infraestrutura própria, que só se torna competitiva quando o *data center* e a equipe já existem.

Como limitações, as premissas de uso não foram medidas em ambiente hospitalar real, a capacidade das consultas na nuvem foi extrapolada a partir de medição local e os preços são referências públicas aproximadas. Como trabalhos futuros, sugerem-se: repetir o cenário C03 com o algoritmo *least outstanding requests*; testar uma mistura realista de operações na AWS; e validar os preços com cotações formais.

REFERÊNCIAS

AMAZON WEB SERVICES. **Amazon CloudFront pricing**. Seattle, 2026d. Disponível em: https://aws.amazon.com/cloudfront/pricing/. Acesso em: 9 nov. 2026.

AMAZON WEB SERVICES. **Amazon S3 pricing**. Seattle, 2026c. Disponível em: https://aws.amazon.com/s3/pricing/. Acesso em: 9 nov. 2026.

AMAZON WEB SERVICES. **AWS Fargate pricing**. Seattle, 2026b. Disponível em: https://aws.amazon.com/fargate/pricing/. Acesso em: 9 nov. 2026.

AMAZON WEB SERVICES. **AWS Pricing Calculator**. Seattle, 2026a. Disponível em: https://calculator.aws/. Acesso em: 9 nov. 2026.

AMAZON WEB SERVICES. **Elastic Load Balancing pricing**. Seattle, 2026e. Disponível em: https://aws.amazon.com/elasticloadbalancing/pricing/. Acesso em: 9 nov. 2026.

BRASIL. Lei nº 12.965, de 23 de abril de 2014. Estabelece princípios, garantias, direitos e deveres para o uso da Internet no Brasil. **Diário Oficial da União**: seção 1, Brasília, DF, 24 abr. 2014. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2011-2014/2014/lei/l12965.htm. Acesso em: 9 nov. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**: seção 1, Brasília, DF, 15 ago. 2018a. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 9 nov. 2026.

BRASIL. Lei nº 13.787, de 27 de dezembro de 2018. Dispõe sobre a digitalização e a utilização de sistemas informatizados para a guarda, o armazenamento e o manuseio de prontuário de paciente. **Diário Oficial da União**: seção 1, Brasília, DF, 28 dez. 2018b. Disponível em: https://www2.camara.leg.br/legin/fed/lei/2018/lei-13787-27-dezembro-2018-787543-publicacaooriginal-157119-pl.html. Acesso em: 9 nov. 2026.

CONSELHO FEDERAL DE MEDICINA. **CFM esclarece regras para descarte de prontuários médicos após 20 anos**. Brasília, DF, 2026. Disponível em: https://portal.cfm.org.br/noticias/cfm-esclarece-regras-para-descarte-de-prontuarios-medicos-apos-20-anos. Acesso em: 9 nov. 2026.

CONSELHO FEDERAL DE MEDICINA. **Resolução CFM nº 1.821, de 11 de julho de 2007**. Aprova as normas técnicas concernentes à digitalização e uso dos sistemas informatizados para a guarda e manuseio dos documentos dos prontuários dos pacientes. Brasília, DF: CFM, 2007.

MONGODB. **MongoDB Atlas pricing**. New York, 2026. Disponível em: https://www.mongodb.com/pricing. Acesso em: 9 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.